arXiv:2504.09775v5 [cs.AR] 20 Mar 2026

End-to-End Modeling and Optimization of Multi-Stage LLM Serving Across the HW/SW Stack

Abhimanyu Rajeshkumar Bambhaniya[1], Hanjiang Wu[1], Suvinay Subramanian[2], Sudarshan Srinivasan[3], Souvik Kundu[4], Amir Yazdanbakhsh[5], Midhilesh Elavazhagan[3], Madhu Kumar[3], Tushar Krishna[1]
[1]Georgia Institute of Technology, [2]Google, [3]Intel, [4]Intel Labs [5]Google DeepMind,
Corresponding email: abambhaniya3@gatech.edu

***Abstract*—Modern LLM serving has moved well beyond the classical prefill–decode pathway, evolving into multi-stage inference pipelines that integrate retrieval, KV-cache lookups, model routing, staged or speculative decoding, and multi-step reasoning. These stages exhibit diverse computational, memory, and communication behaviors, and must be orchestrated across heterogeneous hardware resources, such as GPUs/ASICs, CPUs, multi-level memories, and hierarchical interconnects. As a result, the design space for distributed LLM serving has expanded into a tightly coupled hardware–software co-design problem. Yet the community lacks a framework capable of modeling these multi-stage interactions end-to-end or evaluating how isolated system or hardware decisions propagate through the pipeline to affect latency, throughput, and cost. We present MIST [1], an extensible simulator that jointly models multi-stage AI pipelines, scheduling and batching policies, and detailed hardware effects. MIST enables rigorous exploration of serving strategies across diverse deployments. Using MIST, we uncover up to 2.8× tokens-per-dollar gains for three representative multi-stage LLM workloads and analyze how alternative prefix-KV storage architectural designs impact P99 tail-latency. MIST offers a unified, high-fidelity (< 6% average disparity with deployed inference systems) platform for characterizing, optimizing, and architecting next-generation LLM serving systems, with significantly reduced trial-and-error.**

I. Introduction

Large Language Model (LLM) inference has rapidly become a principal workload in modern computing infrastructure. While training is compute-intensive but predominantly offline, inference must satisfy strict latency, throughput, and cost constraints under highly variable and unpredicted request patterns and diverse deployment environments. These demands arise across the spectrum of LLM applications, from trillion-parameter frontier models used for complex reasoning [22], [57] and agentic workflows [37], [60] to compact models powering chatbots and retrieval-based assistants [19], [40], [44]. As adoption accelerates, the efficiency and robustness of LLM *serving* platforms increasingly determine the usable performance of AI systems at scale.

Modern LLM serving is no longer dominated by the classical prefill–decode path. Production workloads routinely involve *multi-stage* pipelines, combining pre-processing, retrieval-augmented generation (RAG) [27], prefix-KV cache lookups [13], staged or speculative decoding [34], and post-processing (e.g., ranking, validation, or safety filtering). The composition

[1]**M**ulti-stage AI **I**nference **S**imulation **T**oolkit

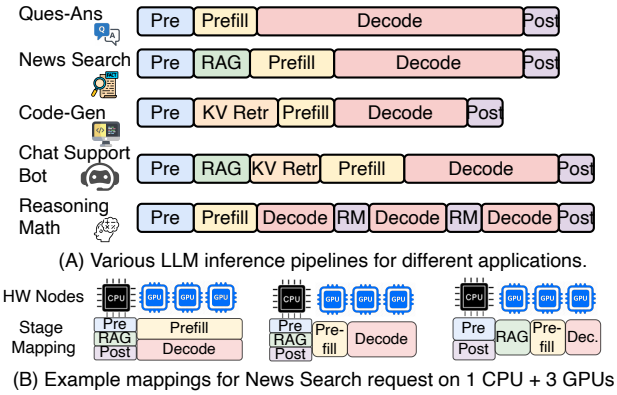
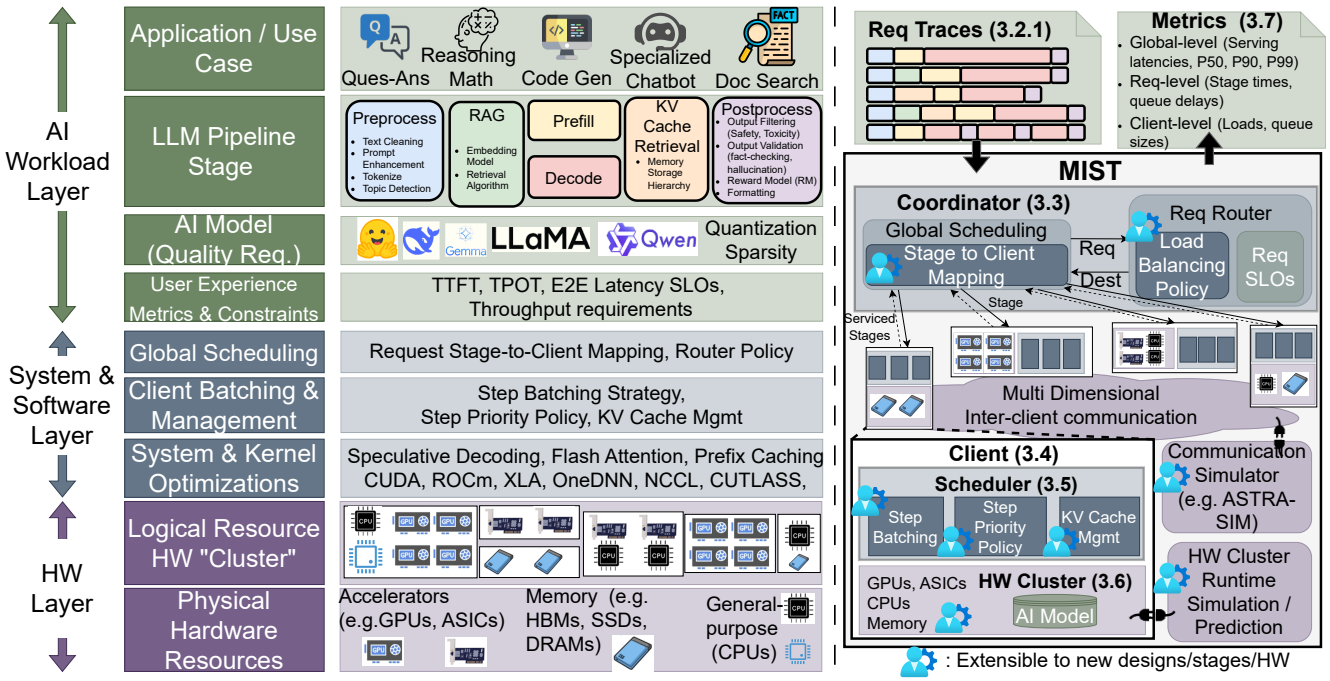


Fig. 1: (a) LLM inference request types: *Question-answering* (Standard); *News search* (RAG pipeline) [17]; *Code generation* (KV cache reuse) [26]; *Chat support* (RAG + KV cache) [16]; and *Reasoning Math* (Multi-turn reasoning + Reward Model) [7]. (b) Scheduling three News Search requests across 1 CPU and 3 GPU nodes.

of these stages varies across use cases, as illustrated in Fig. 1(A), and each stage introduces distinct computational, memory, and communication characteristics. Moreover, stages may themselves be multi-step and may run on heterogeneous devices spanning GPUs, NPUs, CPUs, memory offload layers, and storage. These stages must be orchestrated across clients—logical groupings of hardware resources managed by a shared software stack—as shown in Fig. 1(B). This shift fundamentally alters the nature of LLM serving: end-to-end performance now emerges from *interactions across stages*, across clients, and across layers of the hardware–software stack, not from isolated model execution alone (e.g. prefill-decode path).

As shown in Fig. 2(a), the performance and cost of multi-stage pipelines depend on a deeply entangled set of decisions, such as batching and packing policies, chunk sizes, routing and stage-to-client mapping, parallelization configurations, prefix-KV placement, and memory hierarchy choices. These axes interact non-linearly. For example, accelerating prefill may increase decode queuing delays; routing RAG or KV-retrieval to different clients may shift tail-latency distributions; and the efficacy of prefix-KV reuse depends simultaneously on storage bandwidth, hit rates, and interconnect topology. Even for modest



(a) LLM Inference Serving Stack (b) MIST Overview

Fig. 2: Understanding the LLM inference serving stack and how MIST models the stack. MIST simulates collection of clients. Each client consist of a scheduler which issues steps(e.g. tokens for decode, chunks for prefill, Rerank/Retreival for RAG) to a HW cluster (e.g., Nvidia HGX, AMD MI300X, CPU host with offloading memory instance, ..). The HW cluster is a collection of multiple HW Nodes (e.g. NPUs, memory and CPUs).

device budgets, the combinatorial space yields thousands of valid deployments with very different behaviors. Exhaustive exploration is prohibitively expensive—in one representative case, fully benchmarking just the 3.5K configurations of an 8-GPU deployment across common accelerator types (e.g., A100, H100, etc.) would exceed $51,000 in cloud cost.[2]

Despite the rising complexity of LLM serving, there is *no* systematic methodology for navigating the *end-to-end* co-design space of multi-stage inference pipelines. Existing modeling frameworks [5], [11], [15], [64] share two structural limitations: **(a)** *They model only the prefill and decode stages, omitting RAG, KV retrieval, reward-model evaluation, preprocessing/postprocessing, or stage composition* and **(b)** *they assume a fixed serving architecture—either aggregated or disaggregated—and cannot model cross-client stage routing, multi-client scheduling, or interactions across heterogeneous hardware tiers*. Consequently, current tools cannot capture the cross-stage behaviors that dominate real-world performance. As such, hardware designers lack the means to reason about next-generation architectures; practitioners rely on heuristics, intuition, or expensive trial-and-error.

We argue that characterizing, optimizing, and designing modern LLM serving platforms requires an *end-to-end*, *multi-stage*, *multi-client*, and *HW/SW co-design* simulator. A simulator capable of modeling heterogeneous multi-stage pipelines, complex orchestration policies, and detailed hardware behaviors within a unified framework has been missing in the literature. No existing system provides such end-to-end coverage, nor achieves reasonably high fidelity with real hardware.

To fill this gap, we present MIST, an event-driven, end-to-end simulation framework for multi-stage AI inference pipelines. As shown in Fig. 2(b), MIST accepts pipeline-structured request traces and models how stages flow through the serving stack: from application-level pipelines to client schedulers, network fabric, and hardware clusters. MIST provides:

- Support for diverse stages, including RAG, KV cache retrieval, reasoning, prefill, and decode, along with multi-model pipelines such as dense and MoE models.
- Global routing and load-balancing policies; client-level scheduling; batching and chunking strategies; and KV cache management across heterogeneous clients.
- Modular abstractions for heterogeneous accelerators (GPUs/ASICs/CPUs) with either real-HW plug-ins, ML-based runtime predictors, or cycle-accurate simulators.

Across validated hardware platforms, MIST achieves end-to-end fidelity within 6% of real-system performance.

Using MIST, we answer two classes of previously inaccessible questions:

- We leverage MIST to guide real-system deployments via inference frameworks like vLLM and SGLang, focusing on three representative multi-stage workloads: (1) article retrieval + summarization, (2) code generation with prefix-KV reuse,

[2]This assumes full end-to-end execution for each configuration; see detailed breakdown in Section V-A.

and (3) multi-step mathematical reasoning. As a result, MIST -optimized system yields up to 4.7× tokens/$ improvements over baseline configurations (Sec. V-A).

- We use MIST to evaluate *alternative and emerging* prefix-KV storage architectures, revealing how memory-hierarchy design and cross-node KV mobility shape end-to-end latency (Sec. V-B). Using these detailed analytical results, we identify the optimal memory-architecture design across different KV characteristics (e.g., length and degree of sharing).

In summary, MIST provides the first end-to-end framework for exploring the rapidly expanding design space of AI inference, offering a principled foundation for both optimizing current deployments and architecting alternative AI infrastructures.

II. Background on LLM Inference Pipelines

Fig. 1 showcases the components within the workflow of modern inference. Diverse application use cases demand different pipelines. For example, a news search involves a RAG lookup followed by an auto-regressive prefill and decodel while accurately solving complex mathematical problems typically interleaves generation with reward-model evaluation [7], [36].

A. Pre/ Postprocessing

The **preprocessing** stage of an LLM inference pipeline encompasses a diverse set of transformations that prepare raw user inputs for model consumption. These can be grouped into distinct ***buckets*** based on their computational and latency characteristics: (1) *Text normalization and cleaning*:including lowercasing, Unicode canonicalization, and punctuation normalization; (2) *Linguistic analysis and intent classification*:covering entity extraction, topic detection, and routing decisions; (3) *Model-specific adaptation*:such as tokenization, padding / truncation to match context length, and attention mask construction; and (4) *Prompt Augmentation*:adding system prompts, and prompt enrichment from external knowledge bases. The computational profile of these buckets varies: normalization and classification are typically CPU-bound and scale with input length, and can dominate preprocessing time [24], [35].

Postprocessing stages can similarly be clustered: (1) *Detokenization and text reformatting*:converting model outputs back to natural text or structured formats (JSON, Markdown, tabular data); (2) *Validation and filtering*:toxicity and bias detection using rule-based systems or small classifiers [18]; (3) *Content enhancement*:adding citations, hyperlinking, or summarizing; and (4) *Response scoring*:in reasoning-heavy pipelines, running reward models, such as Outcome Reward Models (ORMs) or Process Reward Models (PRMs) [51] to evaluate output quality. While detokenization is generally lightweight, reward model inference and complex validation modules can introduce GPU-bound latency overheads comparable to the main LLM inference stage.

B. Prefill and Decode

Prefill performs a single forward pass over the input prompts to generate the first token, making it compute-intensive. **Decode** then proceeds auto-regressively, generating tokens sequentially.

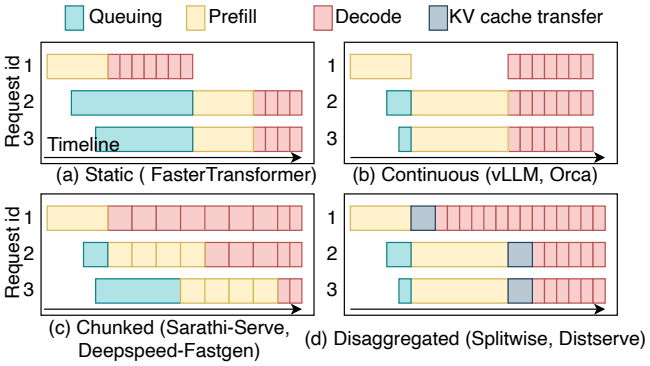


Fig. 3: Batching mechanisms and their latency impact on the prefill and decode phases.

This stage is memory-bound and benefits from batching multiple tokens for higher throughput.

To improve the efficiency and throughput of executing prefill and decode for concurrent requests, there are four primary **batching strategies** (Fig. 3) deployed in practice.

Static Batching requires new requests to wait until the current request fully completes both prefill and decode. As shown in Fig. 3(a), requests 2 and 3 arrive while request 1 is still in prefill, but they cannot proceed until request 1 is entirely served. Only then are the accumulated requests executed together as a batch, adopting a non-preemptive run-to-completion policy.

Continuous Batching boosts compute throughput by prioritizing prefill and then batching decode stages to enhance throughput [33], [63]. As shown in Fig. 3(b), Req 2 and 3 preempt Req 1's decode, and all three decode together after their prefill, improving throughput over static batching .

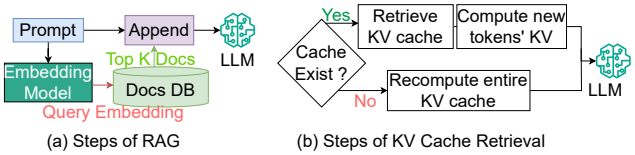
Chunked Batching improves latency-throughput balance by splitting long input sequences into smaller, fixed-size chunks. As shown in Fig. 3(c), this allows prefill (e.g., Req 2) to run alongside decode (e.g., Req 1), avoiding the stalls seen in prefill-prioritized strategies like continuous batching.

Mixed Batching is a combination of chunked and continuous batching. It prioritizes prefill and runs decode in parallel with prefill.

Disaggregated Batching decouples prefill and decode stages by assigning them to independently scaled hardware instances, enabling flexible resource allocation for heterogeneous workloads. In Fig. 3(d), Prefill and Decode run on separate instances using the Continuous mechanism. After Req 1 completes Prefill, it's sent to the Decode instance, freeing the Prefill instance to process Reqs 2 and 3 in parallel.

C. Retrieval Augmented Generation (RAG)

RAG enhances LLM outputs by integrating external knowledge, improving factuality and context. As shown in Fig. 4 (a), it involves two main steps: retrieving relevant documents using embedding models and approximate nearest neighbor (ANN) search such as FAISS [28] and generating responses with the LLM using the augmented prompt. In this work, we adopt Dense Passage Retrieval (DPR) [29] for embedding and use IVF-PQ as our default ANN method, as both are



(a) Steps of RAG (b) Steps of KV Cache Retrieval

Fig. 4: Steps in (a) RAG & (b) KV Retrieval.

widely used techniques that balance memory efficiency and recall. Compared to memory-intensive HNSW [39], IVF clusters vectors into searchable buckets and leverages Product Quantization (PQ) [14] to compress billion-scale DBs [4].

D. KV Cache Retrieval

KV cache retrieval is a key optimization for reducing time-to-first-token (TTFT) in modern inference systems [12], [38], [61] with key steps shown in Fig. 4 (b). Prefix Caching [1] reuses the KV cache from earlier queries when a new query shares a prefix context with them. By bypassing redundant prefill computation, prefix caching substantially lowers both latency and computational overhead. In chat-based applications, KV caches are often persisted across sessions to maintain continuity in multi-turn conversations and create great opportunities for KV cache sharing [61].

E. Reasoning

Reasoning breaks down a complex task into multiple smaller steps, reasoning-based models enable LLMs to generate more accurate answers for problems that require critical thinking and a structured thought process. In real-world applications, this approach typically necessitates multiple iterations of token generation, and evaluating the generated output quality using reward models, such as Outcome Reward Models (ORMs) or Process Reward Models (PRMs) [51]. With each intermediate step refining the reasoning by building upon previous outputs. Reasoning significantly increases computational load and memory requirements, leading to higher end2end latency. In real world application, reasoning can take many forms depending on the model type and prompting strategy. For our analysis, we model the primary dominant contributor to latency, i.e. longer and/or wider output generation chain.

III. MIST: Multi-stage AI Inference Simulation Toolkit

We introduce MIST, a simulation framework designed to capture the complexity of real-world LLM inference pipelines. Fig. 2 shows an overview of the design-space and tool-flow.

A. Overview

To resemble real deployments, MIST simulates the end-to-end execution of state-of-the-art LLM inference pipelines across the three layers of the LLM Inference serving stack shown in Fig. 2(a). Table I defines the terminologies used in MIST .

The **AI Workload Layer** acts as the input to the MIST framework. It comes in the form of a list of *requests* being injected to the framework with different arrival times. A **Request** (Sec. III-B1) is a pipeline of *stages* emulating a target application. Each stage could have one or more *steps* - for instance RAG and KV Cache retrieval (Fig. 4).

TABLE I: Various terminologies used in the paper.

Terminology	Description
Node	Unique HW Entity (E.g. GPUs, ASIC, CPUs, Memory Nodes)
Cluster	Logical Collection of Nodes capable of servicing one or more LLM stage (E.g. 2xH100s, 8xTPUv6, CPU)
Scheduler	Token-level batching + KV Mgmt
Client	Cluster + Scheduler (E.g. vLLM/ SGLang)
Coordinator	Collection of Heterogeneous Clients + Global Req Control (E.g Request Routing to Clients)
Step	Part of a single LLM pipeline stage. (E.g. RAG = Emb. + ReRank + Rtrvl)
Stage	E.g., Prefill, Decode, KV Retrieval, RAG, Preprocess, Postprocess
Request	User input processed through Pipeline of Stages for AI usecase (Fig. 1.(a)).

The **System and Software Layer** layer forms the core of MIST. The flow starts with a **Coordinator** (Sec. III-C) that performs global scheduling of the inference pipeline. Specifically, it maps and routes different stages of the requests to appropriate *clients*, depending on the routing and load balancing policies. Each **Client** (Sec. III-D) employs a runtime **Scheduler** (Sec. III-E) for executing the different steps of the mapped stage over its underlying HW *cluster*. A **Cluster** (Sec. III-F) is a logical collection of the hardware nodes within a physical AI platform [3]. Thus each client incorporating scheduler and HW cluster is similar to real-world LLM frameworks like SGLang [66] and vLLM [33].

The **HW Layer** models the actual physical hardware nodes (GPUs, ASICs, CPUs, memories and their connectivity). In MIST, the HW layer can be simulated at the desired level of fidelity. For runtime simulation users can choose between (i) analytical modeling equation, (ii) empirical observed runtime values from real HW, or (iii) external simulators.

The final outputs (Sec. III-G) of MIST are various LLM performance metrics at the request level, client level, and coordinator level, like TTFTs, TPOTs, queueing delay, etc.

B. Workload Layer: Inputs and Parameters

MIST accepts request traces as input from AI workload layer. In the system and software layer, the router policy, client configuration (Scheduler config + HW clusters), and global client connections as user defined hyper parameters.

1) Request Modeling: MIST currently supports request that are any combination of six stages of the LLM pipeline (***Preprocess***, ***RAG***, ***Prefill***, ***Decode***, ***KV Cache Retrieval***, and ***Postprocess***). Users can construct arbitrary combination of stages as show in Fig. 1 to simulate a modern AI usecases[4].

[3]For e.g., a physical HGX box with 8×H100 GPUs and 2xCPUs can be mapped as two logical clusters each with 4xH100+1CPU, where first cluster is made a prefill client and second is a decode client in a disaggregated setup. Fig. 1B shows three different possible cluster formations from 1 CPU+3 GPU nodes: (i) prefill+decode on the single cluster with 3 GPU, (ii) prefill on cluster with 1 GPU and decode on a cluster with 2 GPU, (iii) RAG, prefill, decode each on a different cluster of 1 GPU.

[4]MIST stages can be extended to model tool calls to study agentic workflows.

Algorithm 1: Coordinator Simulation Algorithm

```
Initialize: Client interconnect topology
Enqueue arrival of all requests (STAGE-PUSH)
while request_serviced < request_accepted do
    Execute next discrete event in queue
    if Event is STAGE-PUSH then
        Dispatch stage to client
        if Client not allotted then
            Client_next ← Router(Request)
        end if
        Client_next.add(Request)
        Enqueue Client to activate next step if idle (CLIENT-STEP)
    else if Event is CLIENT-STEP then
        Process client step and completed requests
        Finished_Requests ← Client.next_step()
        if Client has requests to process then
            Enqueue Client for next step (CLIENT-STEP)
        end if
        for each request finished current stage do
            if request is complete then
                Mark request as serviced
            else
                Client_next ← Router(Request)
                Start client-transfer event
                Enqueue request for next stage (STAGE-PUSH)
            end if
        end for
    end if
end while
```

2) Input Datasets and Workloads: Inference begins with feeding a trace of requests into the system.

Request size: To model diverse prefill and decode token workloads, we use a combination of real and synthetic traces.*Real traces* from production services, such as Azure trace [10] (Conv and Code), capturing realistic input-output token distributions. *Synthetic traces* are generated based on observed characteristics in common workloads. They are modeled as normal distribution with user configurable mean and variance for input and output tokens.

Request injection is modeled using a range of models including uniform, normal, poisson, and bursty distributions. This approach better reflects real-world traffic patterns and enables more robust evaluation of system behavior under diverse operational scenarios.

Additionally each request has additional parameters depending on the associated stages (e.g. RAG request might have required rag algorithm parameters).

C. System SW Layer: Coordinator

The coordinator manages end-to-end inference execution across clients, ensuring ordered stage scheduling and inter-stage communication.

algorithm 1 shows the core simulation loop, which integrates event scheduling (*Client* & *Request* events), routing, and inter-client communication in a unified discrete-event framework. *Request event:* New requests entering the system or a client returns a request after servicing a stage. *Client event:* Scheduling + Execution of the steps for requests assigned to a client. E.g., for a prefill/decode client running on 8xH100 GPU with chunked batching, the client event is scheduling a single chunk batch with assigned prefill and decode batches, and simulating the HW runtime of the scheduled batch. *Client Transfer event:* Communication from client to another based on how the stages are assigned to different clients. E.g. for disaggregated batching, this would be KV cache transfer from prefill to decode client.

1) Routing and Load Balancing: To determine the next client for a given request stage, the coordinator uses a routing module. When multiple clients are capable of executing the same stage, the router uses the user-defined load-balancing policy to distribute work efficiently. We support four routing policies: Round Robin, Least work outstanding, Load-based, Heavy-Light load split [25].

Load in the latter two policies can be defined using various request attributes, such as: i) input context length, ii) current KV cache size (Would matter if the HW cluster have different memory capacity), iii) tokens remaining to be generated[5]. These metrics enable up to nine distinct routing strategies. MIST has a highly modular router API allowing new routing policies to be integrated with minimal effort.

For different AI model instantiations, we assume each request contains metadata specifying the target model. [6] The router can also exploit client placement information to reduce communication costs, especially in disagg serving where large KV caches must be transferred between clients.

2) Global Communication: Once a routing decision is made, the global communication simulator handles data transfers between clients. It estimates communication overhead based on data size and transfer granularity (e.g., full KV cache vs. layer-wise transfer [46]), accounting for transitions between multi-dimensional network hierarchy of HW nodes. For simulating multi-level interconnects, MIST integrates with Astra-Sim [48][7], enabling accurate modeling of communication latency and bandwidth constraints. After data is transferred, the target client can start the stage processing.

D. System SW Layer: Clients

MIST has four different client types: *LLM Client(for Prefill and Decode)*, *RAG client*, *KV Retrieval client*, and *Pre/Post processing client* covering most modern LLM use-cases. Each client in MIST is composed of a runtime Scheduler and a HW Cluster. Drawing inspiration from vLLM [33], each client operates at the granularity of a step. In the case of prefill and decode, a step corresponds to a single forward pass. During the execution of a step, new requests may arrive asynchronously but cannot preempt the ongoing computation. To accommodate different batching strategies shown in Fig. 3, the data size of each step is configurable at the scheduler level (see Sec. III-E.1).

[5]We do not predict number of output tokens, nor do we use it for any of our experiments. There is existing research [25] which can be used to predict number of output tokens.

[6]Future extensions may support adaptive model routing based on request characteristics such as complexity, quality, or priority.

[7]Astra-Sim is network simulator with ability to model multi-dimensional networks in contention-aware fashion [9].

With the scheduled data batch at each step, the runtime and cost are modeled separately for each client type. For example, KV cache retrieval depends on both the size of the KV cache and the details of the hierarchical memory architecture to estimate fetch latencies. Further details on how different hardware cluster are simulated according to different client type is provided in Sec. III-F.

E. System SW Layer: Schedulers

Each client has a scheduler which assigns requests to execute at each step. We define two base scheduler: i) Batched: Used for single step tasks like word lookup. Batching all requests in the client parallely will extract maximum reuse. ii) Sequential: For tasks without reuse possibility, e.g. padding and truncation, etc. We assign available cores to complete the tasks in linear fashion. Pre/post-processing client uses the sequential scheduler while RAG, and KV cache retrieval clients use the batched scheduler to maximize the efficiency.

1) LLM Scheduler: Since LLM inference requires multiple steps to complete the request, it requires a special scheduler. LLM scheduler enforces batching policies and is modeled after vLLM's scheduler. MIST currently supports five batching strategies: *Static Batching* (FasterTransformers [43]) *Continuous Batching* (Orca/vLLM [32], [63]) *Chunked Batching* (Sarathi-Serve, FastGen [6], [23]) *Mixed Batching* (Splitwise Prefill [46]) *Disaggregated Batching* (Splitwise/DistServe [46], [67]).

For each batching strategy, scheduler also supports flexible request packing policies such as *First-Come-First-Serve (FCFS)* and *Least Work Left*. The scheduler and batching APIs are modular, allowing users to define custom packing or scheduling strategies with minimal effort.

In addition to batching policies, the scheduler enforces user-defined constraints such as the maximum number of batched tokens or batch size. Scheduler also manages on-device memory by preventing request admission when memory (e.g., KV cache) is insufficient and by evicting KV caches of completed requests.

F. HW Layer: Cluster Modeling

For each client in MIST , we have a corresponding HW cluster. These clusters are logical groups of HW nodes, and we simulate these through one of the four methods:

- **Real Execution Runtime**: Execute the actual pipeline stage/step on real hardware. E.g., DecodeBatch[Req1, Req2]. We maintain a local database of execution times. If the *same* batch's runtime is needed, we can look it up instead of running again[8].
- **Empirical Runtime**: Profile various combinations of requests on real hardware and create a database of runtimes. Train an ML prediction model to create a real HW runtime database. The key difference from the previous method is that one does not need access to real HW at all times.
- **Analytical Modeling**: We can use empirical runtime-inspired analytical models to model runtime for components with a smaller contribution to overall runtime.

[8]This is very specific to the LLM inference engine, i.e., runtime from vLLM vs SGLang would require unique databases.

TABLE II: Hardware cluster sim currently supported in MIST

Cluster	Real Execution	Empirical Runtime	Analytical Model	External Simulator
LLM	✓	✓	✓	✓
Rag	🛠	✓	✓	✗
KV Retrieval	🛠	✓	✓	✗
Pre/Post Processing	✓	✓	✗	✗

- **External Simulator**: We can plug in an external hardware simulator [31], [41], [45], [47], [58] to get the hardware runtime of unavailable systems.

Table II shows the methods supported for different hardware clusters. We detail the implementation of each hardware cluster in Sec. IV-A

G. Output Metrics

MIST collects detailed metrics during simulation to analyze how requests are processed across the system. These metrics inform performance insights and guide system design decisions. We categorize the collected data as follows:

Individual Request Metrics: For every request, we record fine-grained statistics, including: associated stage metrics (client assignment time, stage start time, stage end time), for prefill and decode, we also maintain each token metrics(scheduled time, token start time and token end time).

Scheduler-Level Metrics: These metrics track the request load queued and processed at each simulation step. This includes: Instantaneous and average queue length, variations in arrival volume scheduling rate, step-wise memory load, and finished requests.

Client-Level Metrics: Each client instance maintains operational statistics through its scheduler. Tracked metrics include: Load and queue size at specific timepoints, Request service rate over time, Estimated power consumption.

Coordinator Metrics: To capture holistic system behavior, we log aggregate statistics such as, serviced requests information, latency breakdowns (mean, P50, P90, P99), and communication metrics.

These global insights enable comprehensive evaluation of system performance and comparative analysis across configurations and scheduling strategies.

Request Tracing and Visualization: All request-level execution details are encoded in JSON format, capturing each stage of processing. This format enables seamless integration with visualization tools, such as Chrome Tracing.

Together, the input datasets and output metrics constitute the foundation of our request modeling pipeline, enabling rigorous, end-to-end analysis of inference system behavior under a wide range of workload conditions.

H. Extending MIST .

MIST has a highly modular and hierarchical design (illustrated in Fig. 2(b)). This allows for the decoupling of AI workload, system & software layer and hardware components, enabling seamless integration across the stack. MIST employs base-class abstractions for pipeline stages and hardware clients, ensuring extensibility as workloads evolve. Adding a new stage only requires specifying its parameters and a latency model (Sec. III-F).

IV. MIST Implementation and Validation

In this section, we provide relevant details of our implementation and validation.

A. Implementation of HW Layer

For the purposes of our case studies (Sec. V), we developed and integrated the following models for the hardware clusters (shown in Table II).

1) LLM Cluster: To model the prefill and decode runtime, in our experiments, we use *empirical runtime* data from hardware clusters(H100, A100, L40S) to predict the runtime of prefill and decode. We collect over 200k datapoints on running vLLM with various. We vary input size, batch size, chunk size (for chunked batching), and tensor parallelism (TP1/TP2/TP4/TP8). We create estimators for each model, hardware, and stage. Using an ensemble of regressors, each trained on a distinct, pre-specified subset of the data rather than stochastic samples. Using this method, we obtain an average error of 2.5% with a median error of less than 1%. This approach is 20–50× faster compared to real execution and is much more cost-effective.

Additionally, MIST also supports *external simulators* (specifically GenZ [11] and LLMCompass [64]) to model hypothetical HW configurations (e.g. Nvidia Rubin [53] or Google Ironwood [54]) to run prefill and decode. MIST also provides *real execution runtime* via vLLM model execution.

2) RAG cluster: RAG runtime consists of i) Converting the input query into a search space embedding, ii) Re-ranking the top k documents, and iii) Retrieving documents. We use the LangChain implementation to collect *empirical runtime* data for running RAG. [9]

3) KV Retrieval: MIST models KV cache retrieval cluster with *analytical model* as a multi-level memory hierarchy, similar in spirit to CPU cache systems. Each level in the hierarchy is characterized by its capacity, lookup latency (ranging from nanoseconds to milliseconds), bandwidth, and cache hit rate. However, unlike CPU caches, where a miss leads to DRAM access, a miss in prefix caching may result in the need to recompute the entire context using the LLM, which is significantly more expensive.

The expected retrieval latency for a cache retrieval request with cache size, is computed recursively using the following expression $T_{\text{retrieval}} = f(\text{Size}_{KV}, C_1)$, where

$$f(\text{KV}, C_n) = \text{H}_n \cdot \left(T_n + \frac{\text{Size}_{KV}}{\text{BW}_n} \right) + (1 - \text{H}_n) \cdot f(\text{Size}_{KV}, C_{n+1})$$

For cache n in cache hierarchy, H_n refers to hit rate of cache, T_n /BW_n refers to the lookup latency and retrieval bandwidth of the cache. at This formulation captures the expected latency by recursively aggregating the time cost of each cache level based on its hit probability.

[9]The RAG retrieval latency is dependent on the number of documents, the retrieval algorithm. For this work we choose the default configuration of langchain running on 64 core x86 CPU.

4) Pre/Post Processing:: MIST simulates pre/post processing stages using *real execution* and *empirical runtime*. To model analysis and intent classification tasks and validation and filtering tasks, we use a forward pass on a small LLM (~2B). For simpler text operations (e.g., normalization, cleaning), latency is modeled as a linear function of input/output sequence length, extrapolated from empirical runtimes. For fixed-latency tasks, such as prompt augmentation, we use empirical runtimes. GPU-bound postprocessing modules like PRMs and ORMs are modeled as a single prefill pass of Qwen2.5-Math-PRM-7B [65].

B. MIST Validation

Next, we demonstrate MIST's fidelity with different cases. We perform three end-to-end validations (Aggregated-Offline, Aggregated-Online, Disaggregated-Online) of MIST on various platforms and models. We also validate the implementation of LLM Cluster and KV retrieval on different hardware.

1) Aggregated-Offline Validation: We validate MIST by running offline workloads where all requests are assumed to have arrived before the system starts, against vLLM runtime on **H100x8** for running LLama3-70B model's performance under varying input lengths, request counts, and chunk sizes. Fig. 5 shows that MIST achieves high fidelity, with less than 2% average error across a range of hardware cluster sizes and serving chunk configurations.

2) Aggregated-Online Validation: We then evaluate MIST using an online workload in which we assume requests arrive with a Poisson distribution at the arrival rate of 5 Queries per second(QPS). Using vLLM, we profile Llama3-70B on **H100x8** and Qwen3-32B on **L40Sx2**, both running the ShareGPT [8] dataset. Fig. 6 shows that the end-to-end runtime of 100 requests matches are with 5.3% error.

3) Disaggregated-Online Validation: To validate the effectiveness of disaggregated client serving, we utilize real request traces collected from the Azure platform [10]. Splitwise [56] implements disaggregated serving atop vLLM for small-scale system evaluation. We compare our implementation against Splitwise-sim as a proxy for a real system, as we lack access to large systems (80 and 160 GPUs).

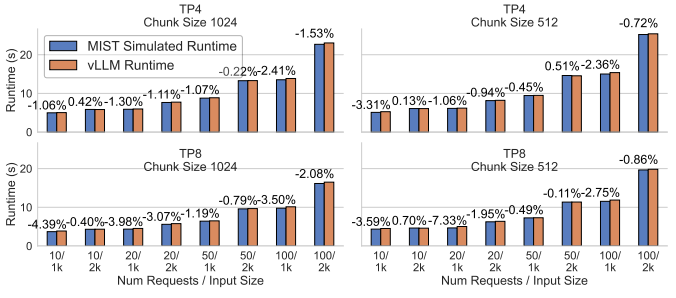


Fig. 5: End-to-end runtime comparison of vLLM real HW runtime and MIST simulated runtime for different parallelization with HGX:H100x8 running Llama3.1-70B. For each hardware configuration, we vary the context length, number of requests, and chunk size.

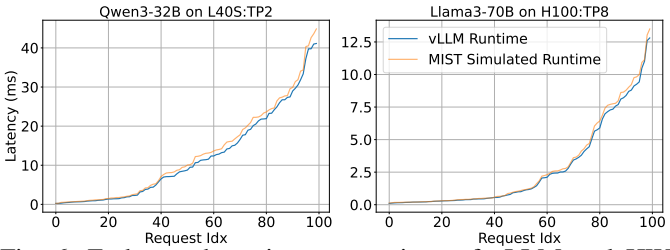


Fig. 6: End-to-end runtime comparison of vLLM real HW runtime vs MIST simulated runtime.

We simulate two different models(Llama-2-70B and Bloom-176B) on an **80xH100** system configuration with 8 prefill clients and 2 decode clients under different request distributions (RPS=20 and RPS=40). Across use cases, we observe minor differences in modeling (< 6% maximum error) as shown in Fig. 7. This major difference in runtime arises from communication, as splitwise-sim employs a dummy link-based communication model with a specified lower-bound bandwidth. In contrast, we use Astra-sim to model client communication, which introduces slight differences in overall runtime. [10]

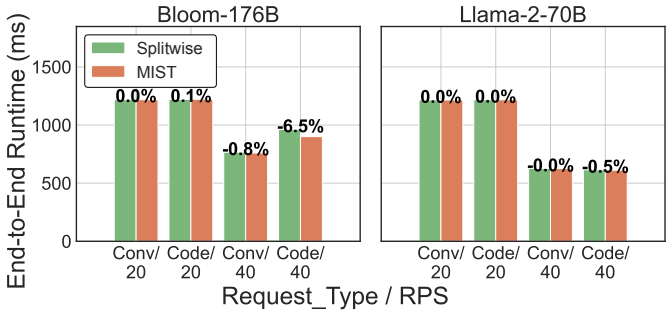


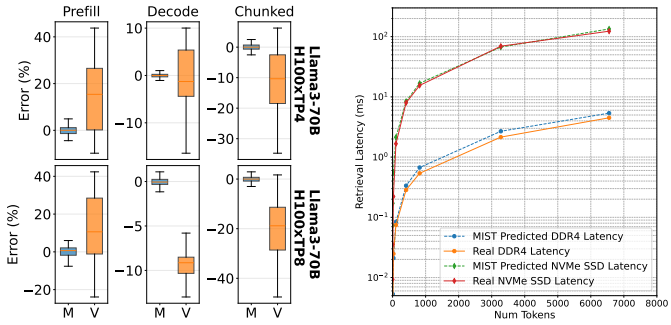
Fig. 7: End-to-end validation results comparing Splitwise and MIST on an 80-GPU system configured with 8TP.

4) LLM Cluster Modeling Validation: To evaluate the accuracy of our ML-Assisted LLM Cluster Modeling (Sec. IV-A1), we compare the MIST per step runtime prediction and Vidur's predicted runtime against the ground truth value from the vLLM running of Llama2-70B on **H100x8**[11]. Fig. 8a compares the error of MIST and Vidur. MIST 's ML-Assisted LLM cluster modeling accuracy is significantly better than Vidur's runtime prediction. Upon further inspection, we identified two main causes of error in Vidur: (i) using operator-level ML predictors results in error accumulating across layers and operators. (ii) ignoring the kernel launch overheads and smaller operations. MIST uses a predictor trained on the LLM engine's model execution time, which accounts for all the CPU and kernel launch overheads and avoids error cascading.

5) KV Retrieval Cluster Modeling Validation: To evaluate the accuracy of our KV retrieval cluster in MIST, we conducted

[10]It should be noted that for this validation, we used splitwise-sim's LLM runtime predictor, as our aim is to show the correctness of request orchestration by the global coordinator.

[11]We also validated MIST across several additional platforms and models. We do not show that data in the interest of space, but will provide it as part of the MIST artifact upon acceptance.



(a) Comparison of MIST :M predicted and Vidur:**V** predicted runtime against vLLM runtime for 1k requests generating 150k tokens.

(b) Validating KV Retrieval cluster against real memory devices. We use Llama3-70B KV cache retrieval as an example.

Fig. 8: Hardware Cluster Validation

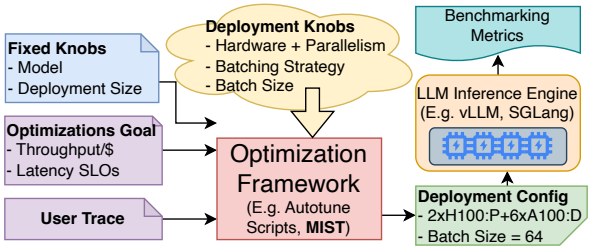


Fig. 9: Optimizing LLM Deployment given a multi-stage use case with LLM Inference Engine on commercial hardware.

a detailed microbenchmark study comparing measured and modeled read latency for both NVMe SSD and DDR4 main memory. Sequential reads were performed using the `fio` benchmarking tool [20] across a range of block sizes, from 256 KB to 1GB. Average read latency was recorded for each size. The experiments were run on a Linux server equipped with 128,GB of DDR4-2666 memory (8 × 16,GB DIMMs, each 64-bit wide, running at 2666,MT/s). The NVMe SSD tests were conducted on an Apple M1 Pro system with a 1,TB Apple AP1024R NVMe SSD, supporting PCIe 4.0 x4. For DDR4, we used an effective peak bandwidth of 150 GB/s and a lookup latency of 80ns, while for NVMe SSD, we assumed a 50 µs lookup latency and 7.0 GB/s.

As shown in Fig. 8b, MIST tracks the trend of measured latency closely across both memory tiers. For DDR4, initial flat latency quickly transitions to a bandwidth-bound regime, while the NVMe SSD shows higher initial overhead and more gradual slope due to lower bandwidth and higher access latency.

V. Evaluation

We demonstrate the value of MIST across two use cases: (i) efficient LLM deployment on current hardware and (ii) future hardware design exploration.

A. MIST for Optimizing LLM Deployment on Current Hardware

In this section, we show how MIST can help efficiently deploy multi-stage LLM use cases with diverse token distributions on off-the-shelf hardware. Modern LLM serving engines [2],

TABLE III: LLM use cases with different multi-stage pipelines.

Usecase	Datasets	Stages	# Queries	# Prefill Tokens			# Decode Tokens		
				Mean	Median	P95	Mean	Median	P95
Article Retrieval and Summarization	Narrative QA [30]	RAG Prefill Decode	47k	19260	11284	59497	218	232	332
Math Reasoning Solution	OpenThoughts [21]	Prefill Reason Decode	114k	158	93	566	6818	5237	17454
Code Generation	Github-Code [49]	KV Retrieval Prefill Decode	465k	26979	16204	83920	1088	329	2753

[33], [66] provide users with a variety of configurable parameters (or tuning knobs) to optimize for efficient deployment. Using MIST, we search for various combinations of these knobs to maximize goodput/sec/dollar (goodput refers to the number of requests meeting the latency SLOs), as shown in Fig. 9.

Search Space: We search the deployment space with the following configurable parameters: hardware SKUs: H100, A100, L40S, model parallelism:TP/PP, replica scheduling:aggregated, disaggregated, client batching:Chunked, Continuous, Mixed, and the number of prefill-to-decode instances in case of disaggregated serving. We compare MIST -suggested deployment configuration baselines, which are vLLM auto-tune [55] run on each hardware SKU, and a brute-force sweep of all combinations[12]. We use AWS per-hour rental costs to calculate the search and deployment costs [13].

We evaluate MIST across three representative multi-stage use cases, Table III. Fig. 11 compares the search cost, best throughput/$, deployment cost of the best configuration for different usecase and models.

1) Optimizing Article Retrieval and Summarization: Use case: Context augmentation is essential for factual accuracy in tasks like document-based QA or legal lookups. We use the Narrative QA dataset (book/movie scripts) as a representative workload, where queries retrieve context to generate answers.

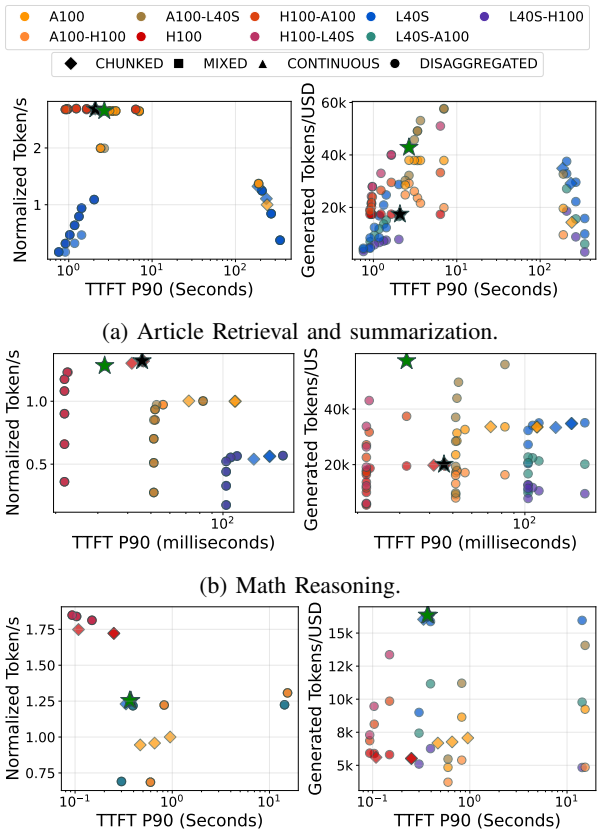
Experimental Setup: We use Qwen3-4B [52] and search deployment configurations (max 8 GPUs) at an 8 RPS injection rate. Given the long context, we enforce TTFT SLOs of P50=2s and P90=10s.

Our search yields 552 valid configurations. Fig. 10a details the throughput of deployments explored by MIST . While an H100 setup (3 prefill, 5 decode, TP1:PP1) achieves the highest throughput due to superior TFLOPS and bandwidth, optimizing for Tokens/s/$ favors a heterogeneous mix: 6 A100 (prefill) and 2 L40S (decode). This configuration generates 2.46× more tokens/$ than the vLLM auto-tune baseline (8xH100 TP1:PP1 with chunked batching).

Insight: RAG

Reducing TTFT has an outsized impact on p99 tail latency. For this purpose, utilizing GPUs for context retrieval can significantly enhance performance.

2) Reasoning-Heavy Deployments: Use case: Reasoning tasks require long output sequences. We use math questions from the OpenThoughts-114K dataset, where prefill length is nominal and decode dominates throughput constraints.



(a) Article Retrieval and summarization.

(b) Math Reasoning.

(c) Code generation with High Prefix-KV Reuse.

Fig. 10: Search-space exploration for different use cases. Throughput is normalized against the Nvidia A100 TP1 x 8 with a chunk size of 2048. Green and Black Star represent the optimal configuration found by MIST and vLLM autotune, respectively.

Experimental Setup: We use Qwen3-14B [52] and search deployment configurations (max 8 GPUs) at 8 RPS. Given the short context, we enforce strict TTFT SLOs: P50/P90 = 50/100 ms.

The search yields 612 valid configurations. Fig. 10b shows the throughput across deployments. While H100 (TP1:PP1, chunked batch size 2048) achieves peak throughput, MIST recommends an H100x1-L40Sx7 configuration. This setup generates 2.85× more tokens/$ than the vLLM auto-tune baseline.

[12]Brute-force enumeration over all hardware and all tuning knobs is prohibitively expensive; we evaluate the brute-force baseline only from a cost perspective, and assume it would

[13]For brute force, cost = $\sum Config_n * Cost$.

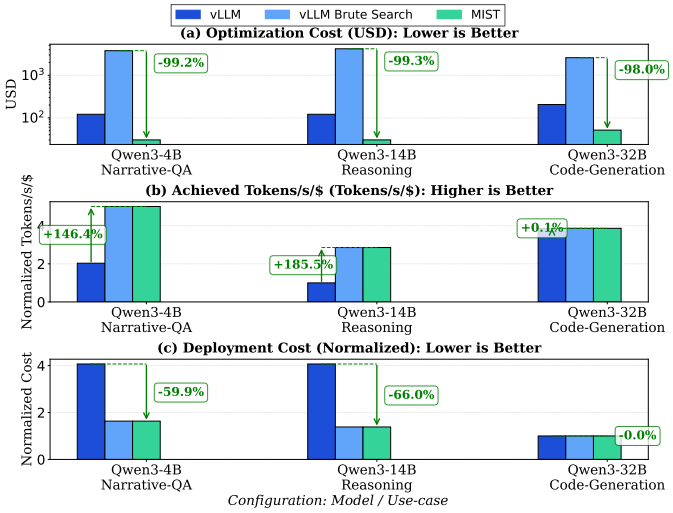


Fig. 11: LLM deployment search results for different models and use-cases.

Insight: Reasoning

While both chunked batching and disaggregated with a large decode pool maintain a low Time Between Tokens (TBT), when a request with a large number of prefill tokens is injected into a chunked batching deployment, the TBT P99 degrades.

3) Code Generation with High Prefix-KV Reuse: Use case: LLM-assisted code-generation has become a common practice in software engineering. For generating code with multi-file dependencies, these dependencies should be provided as context for the prompt. The KV cache of most files can be processed beforehand and reused with prompt [3]. We use the GitHub Code dataset to simulate requests that edit small files using pre-calculated context of other files in the repo.

Experimental Setup: We use Qwen3-32B [52] and search deployment configurations (max 8 GPUs) at 8 RPS. Given the large historical context, we enforce TTFT SLOs of P50=0.75s and P90=2s.

The search yields 378 valid configurations (Fig. 10c). Since most context is pre-calculated, computation on new tokens is minimal. Consequently, a deployment of four clients using L40S (TP2:PP1) with mixed batching yields the best Tokens/s/$.

Insight: KV Retrieval

KV transfer can become a bottleneck in disaggregated serving. **Simultaneously initiating transfer to prefill and decode servers** eliminates sequential delays and balances cross-node communication.

MIST outperforms VLLM autotune reducing deployment costs by up to **66%** and increasing tokens per dollar by up to **185.5%**, with lower search costs.

B. MIST for future architecture design

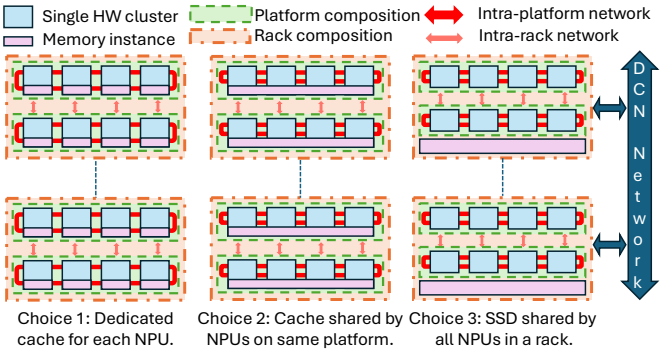


Fig. 12: Design choices for KV cache storage.

Efficient memory cache retrieval is crucial for AI inference engines, where rapid access to context has a direct impact on overall system performance. In this study, we analyze key trade-offs in cache storage granularity, strategies for cache recomputation, and conditions under which transferring data over a Data Center Network (DCN) is warranted.

Target Usecase: The analysis addresses two principal scenarios: *i) private key-value (KV) caches*—designed for individual user contexts(eg. Personal AI chat engines like ChatGPT, Deepseek). These private KV cache can be accessed by future queries from the same user. *ii) Shared KV caches*—often used in multi-user settings (e.g., enterprise AI or shared codebases) to enable multiple users to access a large corpus ($O(10^{10})$ tokens) of documents or code. Generally, these caches have some KV cache hotspots that would be accessed at a much higher frequency than the rest of the KV cache.

Hardware design space: A typical AI serving architecture with storage memory can comprise various cache storage solutions. We classify these into three choices (Fig. 12): (1) a dedicated cache per client, (2) a platform-level shared cache with shared access by 2-8 clients, and (3) a rack-level shared cache with shared access by 32-64 clients Each configuration presents distinct trade-offs in terms of capacity, bandwidth, and latency.

Experimental Setup: We evaluate these architecture choices on a high-performance cluster comprising 256 GPUs (128×H100:TP2 nodes) distributed across 4 racks, interconnected via NVLink and PCIe. The experiments simulate short (4K tokens) and long (24K tokens) KV cache retrieval workloads for both private and shared contexts. Requests follow the AzureConv trace [10], injected at 240 req/s using a Poisson distribution. We examine five distinct storage architectures, detailed in Table V, ranging from dedicated local memory to a baseline of full recomputation. End-to-end request serving latency serves as the primary evaluation metric to assess system performance across these configurations.

TABLE IV: Comparison of prior works for modeling LLM inference against this work.

Framework	LLM Serving Pipeline Modeling						
	AI Workload Modeling		System & Software Modeling		HW Modeling		
	Pipeline Stages	Models supported	Global Routing Policy	Client Batching Type	Cluster Types	Runtime Simulation	Memory Hierarchy
LLMCompass [64]	Prefill, Decode	Single Model	None	Static Only	Single Client	Analytical simulator for future HW modeling	Single-Level
GenZ [11]	Prefill, Decode	Single Model	None	Static, Continuous, Chunked	Single Client	Analytical + External Simulator for future HW modeling	Multi-Level + Offload)
Vidur [5]	Prefill, Decode	Single Model	Round-Robin Random, Least outstanding	Static, Continuous, Chunked	GPU Pool	Real HW data + ML prediction	Single-Level
LLMServingSim [15]	Prefill, Decode	Single Model	Round-Robin	Continuous Only	NPU Pool + CIMs Pool	Roofline from astra-sim	Multi-Level + Offload)
Splitwise-sim [42]	Prefill, Decode	Single Model	Random, Round-Robin Least outstanding	Global Disaggregated	Three Pools	Real HW data + ML prediction	Single-Level
MIST (ours)	**Cache Retrieval**, **RAG**, Prefill, **Reasoning**, Decode	**Multiple Simultaneous Models**	Random, Round-Robin, Least outstanding, **Load-based**, **Heavy-Light split**	Static, Continous, Chunked, **Global/Local Disaggregated**	**Multiple Pools of varying Nodes**	Real HW data or external simulators + ML prediction	Multi-Level + Offload)

TABLE V: Storage Architecture Configurations. Bandwidth (BW) represents the speed of data retrieval or transfer.

Case	Architecture Type	Capacity / BW	Access / Notes
A	Dedicated Per-Client (Choice 1)	1 TB / 128 GB/s	LPDDR-based; Local access
B	Platform-Shared (Choice 2)	4 TB / 32 GB/s	Shared by 4 clients
C	Rack-Shared (Choice 3)	32 TB / 2 GB/s	Shared by 32 clients
D	Rack-Shared (Choice 3) + DCN	32 TB / 2 GB/s	Inter-rack transfer @ 128GB/s
E	No Cache	N/A	Full KV recomputation

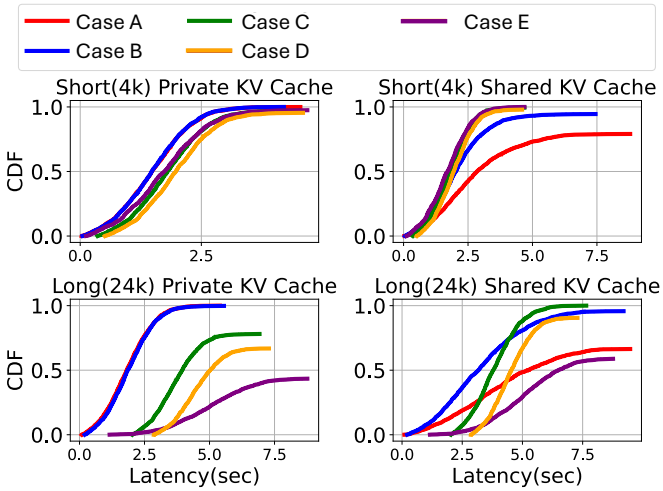


Fig. 13: Comparing different platform architectures for storing past cache storage. Serving 128 clients of Llama-3.1-70B using 4 HGX Racks(64xH100 per Rack).

Fig. 13 shows the end-2-end latency cdf for the design space. For private KV caches, we find that the platform-level shared cache (Case B) offers the best request latency P90. Conversely, for shared global KV caches, a rack-level shared cache (Case C) is superior, as it delivers higher aggregate capacity and maintains acceptable performance despite a modest reduction in per-client bandwidth.

For short KV caches (~4K tokens), Case E (Cache Recomputation) appears to be the best case due to the low overhead associated with recomputation, making it a competitive alternative to direct cache retrieval, particularly when it avoids the additional delay introduced by DCN transfers. However, as the KV cache size increases (24K tokens), the recomputation overhead becomes prohibitive; in such cases, utilizing a rack-level cache to directly retrieve stored data proves to be more efficient. Although Case D, DCN transfers config C can serve as a fallback mechanism in instances of replica overload, the inherent link latency (approximately 20 msec) renders this approach less attractive for large caches.

- Platform-level shared cache (B) is great for private KV caches as it balances speed and resource sharing.
- Rack-level shared cache (C) is optimal for shared global KV caches: Provides low-latency access and efficient inter-node sharing.
- Recomputation (E) is a viable strategy for short shared KV caches especially when cache reuse is limited.

VI. Related Work

AI Training Simulators. Several prior works leverage the predictability of DNN training iterations [48], [50], [59], [62], [68] to model training runtime. While LLM training frameworks are solely focuses on throughput and parallelism across massive datasets, LLM inference requires low-latency, cost-efficient execution with usecase specific optimizations (KV caching, quantization, speculative decoding) and have multi-stage pipelines spanning preprocessing, RAG, prefill, decode, and postprocessing, thus creating separate set of challenges to simulate the inference pipeline.

LLM Inference Simulators. Table IV compares various frameworks simulating the LLM inference serving stack. LLMCompass [64] and GenZ [11] provide detailed modeling capabilities with optimizations but are limited to single-client configurations. Vidur [5] supports multi-client simulation but assumes same type of client and is restricted to modeling real hardware and aggregated batching strategies only. It lacks support for diverse clients, disaggregated hardware for prefill and decode stages. Similarly, LLMServingSim [15] does not support chunked prefill or disaggregated batching, which are increasingly common in production-grade LLM deployments [2]. Splitwise-sim [42] models three pools for hardware clients representing prefill, decode and mixed pool. Similar to LLMServingsim it doesn't model chunked batching. Most importantly, Vidur, LLMServingSim and Splitwise-sim fall short in modeling advanced, multi-stage LLM inference pipelines. *In contrast, MIST is the first simulator designed to support end-to-end modeling of real-world LLM inference pipelines across diverse HW configurations.*

VII. Conclusion

Modern AI serving pipelines demand tools that can optimize across the software-hardware co-design space of complex, multi-stage workflows. We present MIST, a high-fidelity, event-driven simulation framework that captures the full spectrum of inference stages and its corresponding scheduling stack across diverse hardware setups. MIST supports flexible batching, multi-stage execution, and detailed HW modeling, enabling accurate evaluation of architectural trade-offs. We demonstrate how MIST can be useful in optimizing LLM deployments and assist architects in designing better systems for LLM Inference. Looking ahead, MIST can be extended for exploring optimal configurations of future HW, developing adaptive schedulers, and simulating multi-agent LLM deployments.

References

[1] "Automatic prefix caching — vllm," [Online; accessed 2025-04-11]. [Online]. Available: https://docs.vllm.ai/en/latest/design/v1/prefix_caching.html

[2] "Dynamo inference framework — nvidia developer," [Online; accessed 2025-04-10]. [Online]. Available: https://developer.nvidia.com/dynamo

[3] "What is prefix caching? a beginner's guide - ai resources," [Online; accessed 2025-04-11]. [Online]. Available: https://www.modular.com/ai-resources/what-is-prefix-caching-a-beginner-s-guide

[4] "Choose the k-nn algorithm for your billion-scale use case with opensearch — aws big data blog," 9 2022, [Online; accessed 2025-04-11]. [Online]. Available: https://aws.amazon.com/blogs/big-data/choose-the-k-nn-algorithm-for-your-billion-scale-use-case-with-opensearch/

[5] A. Agrawal, N. Kedia, J. Mohan, A. Panwar, N. Kwatra, B. Gulavani, R. Ramjee, and A. Tumanov, "Vidur: A large-scale simulation framework for llm inference," 2024. [Online]. Available: https://arxiv.org/abs/2405.05465

[6] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," *Proceedings of 18th USENIX Symposium on Operating Systems Design and Implementation, 2024, Santa Clara*, 2024.

[7] J. Ahn, R. Verma, R. Lou, D. Liu, R. Zhang, and W. Yin, "Large language models for mathematical reasoning: Progresses and challenges," 2024. [Online]. Available: https://arxiv.org/abs/2402.00157

[8] anon823116, "Sharegpt_vicuna datasets at hugging face," 2024. [Online]. Available: https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered

[9] Astra-Sim, "ns-3 network backend — astra-sim 2.2 documentation," [Online; accessed 2025-06-19]. [Online]. Available: https://astra-sim.github.io/astra-sim-docs/network-backend/ns3-network-backend.html

[10] M. Azure, "Azure Public Dataset: Azure LLM Inference Trace 2023," https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2023.md, 2023, accessed: 2025-04-10.

[11] A. Bambhaniya, R. Raj, G. Jeong, S. Kundu, S. Srinivasan, M. Elavazhagan, M. Kumar, and T. Krishna, "Demystifying platform requirements for diverse llm inference use cases," *arXiv preprint arXiv:2406.01698*, 2024.

[12] Y. Cheng, K. Du, J. Yao, and J. Jiang, "Do large language models need a content delivery network?" *arXiv preprint arXiv:2409.13761*, 2024.

[13] Y. Cheng, Y. Liu, J. Yao, Y. An, X. Chen, S. Feng, Y. Huang, S. Shen, K. Du, and J. Jiang, "Lmcache: An efficient kv cache layer for enterprise-scale llm inference," 2025. [Online]. Available: https://arxiv.org/abs/2510.09665

[14] A. Chirkin, "Accelerating vector search: Nvidia cuvs ivf-pq part 1, deep dive — nvidia technical blog," 7 2024, [Online; accessed 2025-03-08]. [Online]. Available: https://developer.nvidia.com/blog/accelerating-vector-search-nvidia-cuvs-ivf-pq-deep-dive-part-1/

[15] J. Cho, M. Kim, H. Choi, G. Heo, and J. Park, "Llmservingsim: A hw/sw co-simulation infrastructure for llm inference serving at scale," 2024. [Online]. Available: https://arxiv.org/abs/2408.05499

[16] S. K. Dam, C. S. Hong, Y. Qiao, and C. Zhang, "A complete survey on llm-based ai chatbots," 2024. [Online]. Available: https://arxiv.org/abs/2406.16937

[17] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," 2024. [Online]. Available: https://arxiv.org/abs/2312.10997

[18] S. Gehman, S. Gururangan, M. Sap, Y. Choi, and N. A. Smith, "Realtoxicityprompts: Evaluating neural toxic degeneration in language models," 2020. [Online]. Available: https://arxiv.org/abs/2009.11462

[19] Google, "Introducing gemini: Google's most capable ai model yet," 2023. [Online]. Available: https://blog.google/technology/ai/google-gemini-ai/

[20] Google Cloud, "Benchmark Persistent Disk performance on a Linux VM," https://cloud.google.com/compute/docs/disks/benchmarking-pd-performance-linux, 2025, last updated: 2025-08-07; Accessed: 2025-08-20.

[21] E. Guha, R. Marten, S. Keh, N. Raoof, G. Smyrnis, H. Bansal, M. Nezhurina, J. Mercat, T. Vu, Z. Sprague, A. Suvarna, B. Feuer, L. Chen, Z. Khan, E. Frankel, S. Grover, C. Choi, N. Muennighoff, S. Su, W. Zhao, J. Yang, S. Pimpalgaonkar, K. Sharma, C. C.-J. Ji, Y. Deng, S. Pratt, V. Ramanujan, J. Saad-Falcon, J. Li, A. Dave, A. Albalak, K. Arora, B. Wulfe, C. Hegde, G. Durrett, S. Oh, M. Bansal, S. Gabriel, A. Grover, K.-W. Chang, V. Shankar, A. Gokaslan, M. A. Merrill, T. Hashimoto, Y. Choi, J. Jitsev, R. Heckel, M. Sathiamoorthy, A. G. Dimakis, and L. Schmidt, "Openthoughts: Data recipes for reasoning models," 2025. [Online]. Available: https://arxiv.org/abs/2506.04178

[22] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, and Z. Hu, "Reasoning with language model is planning with world model." [Online]. Available: http://arxiv.org/abs/2305.14992

[23] C. Holmes, M. Tanaka, M. Wyatt, A. A. Awan, J. Rasley, S. Rajbhandari, R. Y. Aminabadi, H. Qin, A. Bakhtiari, L. Kurilenko *et al.*, "Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference," *arXiv preprint arXiv:2401.08671*, 2024.

[24] G. Izacard, P. Lewis, M. Lomeli, L. Hosseini, F. Petroni, T. Schick, J. Dwivedi-Yu, A. Joulin, S. Riedel, and E. Grave, "Atlas: Few-shot learning with retrieval augmented language models," *Journal of Machine Learning Research*, vol. 24, no. 251, pp. 1–43, 2023.

[25] K. Jain, A. Parayil, A. Mallick, E. Choukse, X. Qin, J. Zhang, Íñigo Goiri, R. Wang, C. Bansal, V. Rühle, A. Kulkarni, S. Kofsky, and S. Rajmohan, "Intelligent router for llm workloads: Improving performance through workload-aware load balancing," 2025. [Online]. Available: https://arxiv.org/abs/2408.13510

[26] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A survey on large language models for code generation," 2024. [Online]. Available: https://arxiv.org/abs/2406.00515

[27] W. Jiang, S. Subramanian, C. Graves, G. Alonso, A. Yazdanbakhsh, and V. Dadu, "Rago: Systematic performance optimization for retrieval-augmented generation serving," 2025. [Online]. Available: https://arxiv.org/abs/2503.14649

[28] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," 2017. [Online]. Available: https://arxiv.org/abs/1702.08734

[29] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," 2020. [Online]. Available: https://arxiv.org/abs/2004.04906

[30] T. Kočiský, J. Schwarz, P. Blunsom, C. Dyer, K. M. Hermann, G. Melis, and E. Grefenstette, "The NarrativeQA reading comprehension challenge," *Transactions of the Association for Computational Linguistics*, vol. 6, pp. 317–328, 2018. [Online]. Available: https://aclanthology.org/Q18-1023

[31] H. Kwon, P. Chatarasi, V. Sarkar, T. Krishna, M. Pellauer, and A. Parashar, "Maestro: A data-centric approach to understand reuse, performance, and hardware cost of dnn mappings," *IEEE Micro*, vol. 40, no. 3, pp. 20–29, 2020.

[32] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 611–626. [Online]. Available: https://doi.org/10.1145/3600006.3613165

[33] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[34] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," 2023. [Online]. Available: https://arxiv.org/abs/2211.17192

[35] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[36] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," 2023. [Online]. Available: https://arxiv.org/abs/2305.20050

[37] C. Lin, Z. Han, C. Zhang, Y. Yang, F. Yang, C. Chen, and L. Qiu, "Parrot: efficient serving of llm-based applications with semantic variable," in *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'24. USA: USENIX Association, 2024.

[38] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan *et al.*, "Cachegen: Kv cache compression and streaming for fast large language model serving," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 38–56.

[39] J. Mazanec and O. Hamzaoui, "Choose the k-nn algorithm for your billion-scale use case with opensearch — aws big data blog," 9 2022, [Online; accessed 2025-03-08]. [Online]. Available: https://aws.amazon.com/blogs/big-data/choose-the-k-nn-algorithm-for-your-billion-scale-use-case-with-opensearch/

[40] MIcrosoft, "Github copilot · your ai pair programmer." [Online]. Available: https://github.com/features/copilot

[41] N. Muralimanohar, R. Balasubramonian, and N. P. Jouppi, "Cacti 6.0: A tool to model large caches," *HP laboratories*, vol. 27, p. 28, 2009.

[42] Mutinifni, "SplitwiseSim: LLM Serving Cluster Simulator," https://github.com/Mutinifni/splitwise-sim, 2024, accessed: 2025-04-10.

[43] NVIDIA, "Github - nvidia/fastertransformer: Transformer related optimization, including bert, gpt," [Online; accessed 2025-04-10]. [Online]. Available: https://github.com/NVIDIA/FasterTransformer

[44] OpenAI, "Chatgpt." [Online]. Available: https://openai.com/chatgpt

[45] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A systematic approach to dnn accelerator evaluation," in *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019, pp. 304–315.

[46] P. Patel, E. Choukse, C. Zhang, Íñigo Goiri, A. Shah, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative llm inference using phase splitting," 2023.

[47] R. Raj, S. Banerjee, N. Chandra, Z. Wan, J. Tong, A. Samajdar, and T. Krishna, "Scale-sim v3: A modular cycle-accurate systolic accelerator simulator for end-to-end system analysis," 2025. [Online]. Available: https://arxiv.org/abs/2504.15377

[48] S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "ASTRA-SIM: Enabling sw/hw co-design exploration for distributed dl training platforms," in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2020.

[49] N. Saga, "nick007x/github-code-2025· datasets at hugging face," 10 2025, [Online; accessed 2025-11-17]. [Online]. Available: https://huggingface.co/datasets/nick007x/github-code-2025

[50] M. Sivathanu, T. Chugh, S. S. Singapuram, and L. Zhou, "Astra: Exploiting predictability to optimize deep learning," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 909–923. [Online]. Available: https://doi.org/10.1145/3297858.3304072

[51] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling llm test-time compute optimally can be more effective than scaling model parameters," 2024. [Online]. Available: https://arxiv.org/abs/2408.03314

[52] Q. Team, "Qwen3 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[53] C. to Wikimedia projects, "Rubin (microarchitecture) - wikipedia," 6 2024, [Online; accessed 2025-04-11]. [Online]. Available: https://en.wikipedia.org/wiki/Rubin_(microarchitecture)

[54] A. Vahdat, "Ironwood: The first google tpu for the age of inference," 4 2025, [Online; accessed 2025-04-11]. [Online]. Available: https://blog.google/products/google-cloud/ironwood-tpu-age-of-inference/

[55] vllm, "Vllm auto_tune," August 2025, [Online; accessed 2025-11-18]. [Online]. Available: https://github.com/vllm-project/vllm/blob/main/benchmarks/auto_tune/README.md

[56] vLLM contributors, "Add Splitwise Implementation to vLLM," https://github.com/vllm-project/vllm/pull/2809, 2024, accessed: 2025-04-10.

[57] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," 2023. [Online]. Available: https://arxiv.org/abs/2201.11903

[58] W. Won, T. Heo, S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "Astra-sim2. 0: Modeling hierarchical networks and disaggregated systems for large-model training at scale," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2023, pp. 283–294.

[59] W. Xiao, R. Bhardwaj, R. Ramjee, M. Sivathanu, N. Kwatra, Z. Han, P. Patel, X. Peng, H. Zhao, Q. Zhang, F. Yang, and L. Zhou, "Gandiva: Introspective cluster scheduling for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. Carlsbad, CA: USENIX Association, Oct. 2018, pp. 595–610. [Online]. Available: https://www.usenix.org/conference/osdi18/presentation/xiao

[60] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "Swe-agent: Agent-computer interfaces enable automated software engineering," 2024. [Online]. Available: https://arxiv.org/abs/2405.15793

[61] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, "Cacheblend: Fast large language model serving with cached knowledge fusion," *arXiv preprint arXiv:2405.16444*, 2024.

[62] G. X. Yu, Y. Gao, P. Golikov, and G. Pekhimenko, "A runtime-based computational performance predictor for deep neural network training," 2021. [Online]. Available: https://arxiv.org/abs/2102.00527

[63] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for {Transformer-Based} generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.

[64] H. Zhang, A. Ning, R. Prabhakar, and D. Wentzlaff, "A hardware evaluation framework for large language model inference," *arXiv preprint arXiv:2312.03134*, 2023.

[65] Z. Zhang, C. Zheng, Y. Wu, B. Zhang, R. Lin, B. Yu, D. Liu, J. Zhou, and J. Lin, "The lessons of developing process reward models in mathematical reasoning," *arXiv preprint arXiv:2501.07301*, 2025.

[66] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "Sglang: Efficient execution of structured language model programs," 2024. [Online]. Available: https://arxiv.org/abs/2312.07104

[67] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving," 2024.

[68] H. Zhu, A. Phanishayee, and G. Pekhimenko, "Daydream: Accurately estimating the efficacy of optimizations for DNN training," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, Jul. 2020, pp. 337–352. [Online]. Available: https://www.usenix.org/conference/atc20/presentation/zhu-hongyu